

Отчёт по лабораторной работе №13

**Программирование в командном процессоре ОС UNIX. Ветвления
и циклы**

Иванова Анастасия Сергеевна

Содержание

1	Цель работы	4
2	Выполнение лабораторной работы	5
2.1	Задание 1. Скрипт с ключами getopts	5
2.2	Задание 2. Программа на С и анализ кода завершения	9
2.3	Задание 3. Создание и удаление нумерованных файлов	12
2.4	Задание 4. Архивация свежих файлов	15
3	Контрольные вопросы	18
4	Вывод	20

Список иллюстраций

2.1	Создание файла search.sh	5
2.2	Листинг search.sh	8
2.3	Создание test.txt	8
2.4	Результаты работы search.sh	9
2.5	Код checknum.c	10
2.6	Компиляция checknum.c	11
2.7	Листинг run_check.sh	11
2.8	Листинг file_manager.sh	14
2.9	Создание файлов	14
2.10	Удаление файлов	15
2.11	Листинг backup_recent.sh	16
2.12	Подготовка testdir	16
2.13	Архивация и проверка	17

1 Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

2 Выполнение лабораторной работы

2.1 Задание 1. Скрипт с ключами getopt

Разработаем скрипт `search.sh`, который анализирует командную строку с ключами `-inputfile`, `-outputfile`, `-p`, `-C`, `-n` и выполняет поиск строк в указанном файле.

1. Создадим файл скрипта `search.sh` (рис. 2.1):

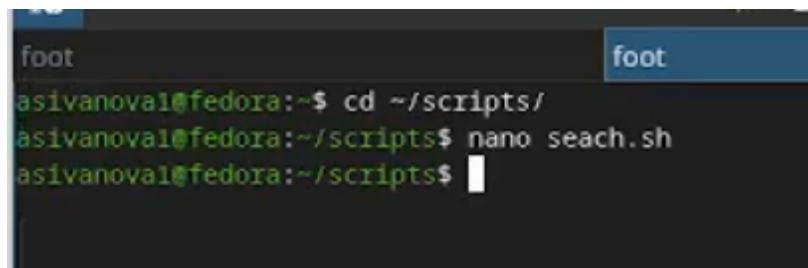


Рисунок 2.1: Создание файла `search.sh`

2. Листинг скрипта `search.sh` (рис. 2.2):

```
#!/bin/bash

input_file=""
output_file=""
pattern=""
```

```

case_sensitive="no"
number_lines="no"

usage() {
    echo "Использование: $0 -inputfile <файл> -p <шаблон> [-outputfile <файл>] [-C] [-n]"
    exit 1
}

while getopts ":i:o:p:Cn" opt; do
    case $opt in
        i) input_file="$OPTARG" ;;
        o) output_file="$OPTARG" ;;
        p) pattern="$OPTARG" ;;
        C) case_sensitive="yes" ;;
        n) number_lines="yes" ;;
        \?) echo "Неизвестный параметр: -$OPTARG" >&2; usage ;;
        :) echo "Параметр -$OPTARG требует аргумента" >&2; usage ;;
    esac
done

if [ -z "$input_file" ] || [ -z "$pattern" ]; then
    echo "Ошибка: необходимо указать -inputfile и -p"
    usage
fi

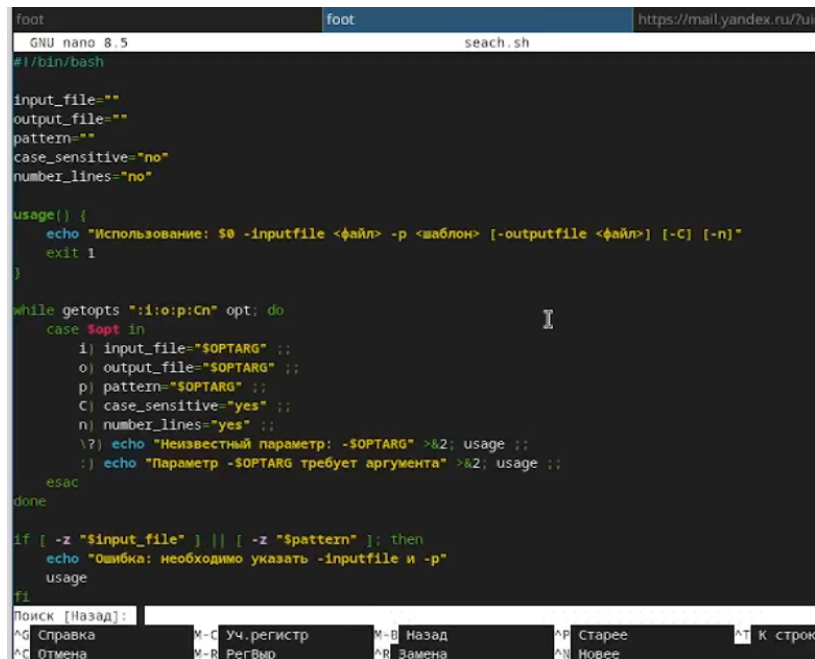
if [ ! -f "$input_file" ]; then
    echo "Ошибка: файл $input_file не существует"
    exit 1

```

```
fi

grep_opts=""
if [ "$case_sensitive" != "yes" ]; then
    grep_opts="-i"
fi
if [ "$number_lines" = "yes" ]; then
    grep_opts="$grep_opts -n"
fi

if [ -n "$output_file" ]; then
    grep $grep_opts "$pattern" "$input_file" > "$output_file"
    echo "Результат сохранён в $output_file"
else
    grep $grep_opts "$pattern" "$input_file"
fi
```



```
foot GNU nano 8.5 search.sh
#!/bin/bash

input_file=""
output_file=""
pattern=""
case_sensitive="no"
number_lines="no"

usage() {
    echo "Использование: $0 -inputfile <файл> -p <шаблон> [-outputfile <файл>] [-C] [-n]"
    exit 1
}

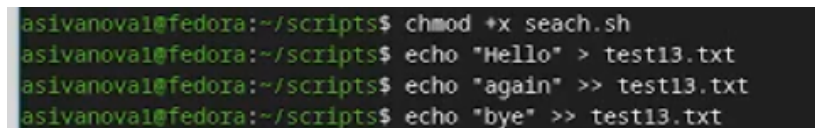
while getopts "i:o:p:Cn" opt; do
    case $opt in
        i) input_file="$OPTARG" ;;
        o) output_file="$OPTARG" ;;
        p) pattern="$OPTARG" ;;
        C) case_sensitive="yes" ;;
        n) number_lines="yes" ;;
        \?) echo "Неизвестный параметр: -$OPTARG" >&2; usage ;;
        :) echo "Параметр -$OPTARG требует аргумента" >&2; usage ;;
    esac
done

if [ -z "$input_file" ] || [ -z "$pattern" ]; then
    echo "Ошибка: необходимо указать -inputfile и -p"
    usage
fi
```

Рисунок 2.2: Листинг search.sh

3. Сделаем файл исполняемым и подготовим тестовый файл test.txt (рис. 2.3):

```
echo "Hello world" > test.txt
echo "hello again" >> test.txt
echo "HELLO upper" >> test.txt
echo "goodbye" >> test.txt
```

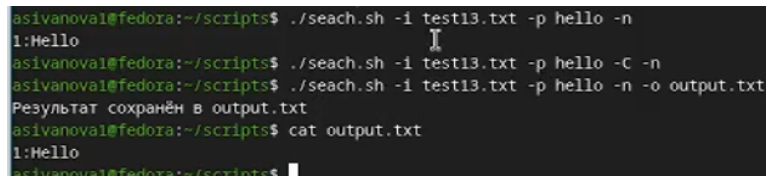


```
asivanoval@fedora:~/scripts$ chmod +x seach.sh
asivanoval@fedora:~/scripts$ echo "Hello" > test13.txt
asivanoval@fedora:~/scripts$ echo "again" >> test13.txt
asivanoval@fedora:~/scripts$ echo "bye" >> test13.txt
```

Рисунок 2.3: Создание test.txt

4. Запустим скрипт с различными ключами (рис. 2.4):


```
./search.sh -i test.txt -p hello -n
./search.sh -i test.txt -p hello -C -n
./search.sh -i test.txt -p hello -n -o output.txt
cat output.txt
```

A screenshot of a terminal window showing the execution of the search.sh script. The user runs the script with various flags: first with -i test13.txt -p hello -n, which outputs '1:Hello'; then with -i test13.txt -p hello -C -n; then with -i test13.txt -p hello -n -o output.txt, which outputs 'Результат сохранён в output.txt'; and finally with cat output.txt, which outputs '1:Hello'.

```
asivanova@fedora:~/scripts$ ./search.sh -i test13.txt -p hello -n
1:Hello
asivanova@fedora:~/scripts$ ./search.sh -i test13.txt -p hello -C -n
asivanova@fedora:~/scripts$ ./search.sh -i test13.txt -p hello -n -o output.txt
Результат сохранён в output.txt
asivanova@fedora:~/scripts$ cat output.txt
1:Hello
asivanova@fedora:~/scripts$
```

Рисунок 2.4: Результаты работы search.sh

2.2 Задание 2. Программа на С и анализ кода завершения

Создадим программу на С checksum.c, которая вводит число и возвращает код завершения в зависимости от его знака. Затем напишем скрипт, анализирующий этот код.

1. Листинг checksum.c (рис. 2.5):

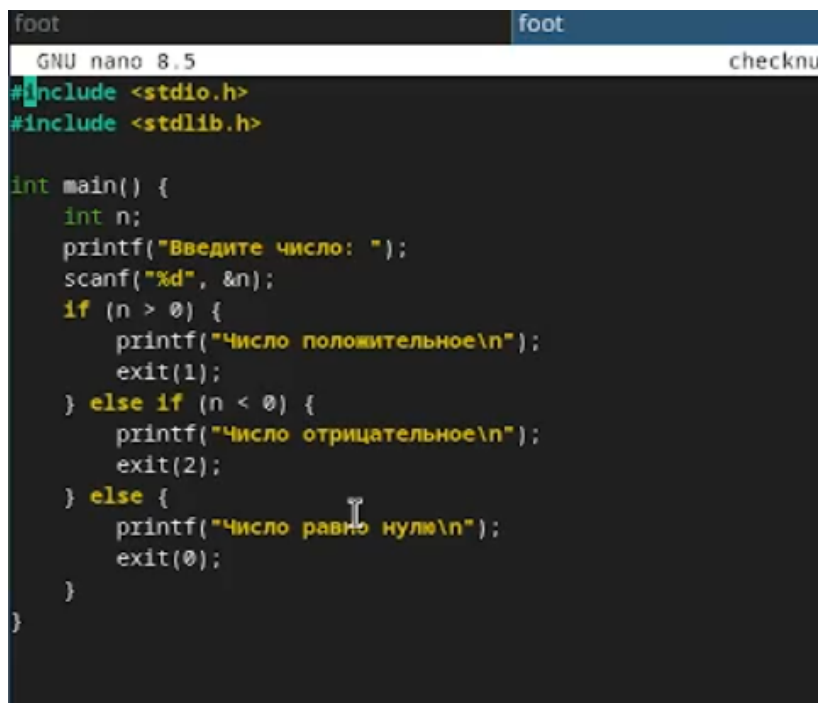
```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    printf("Введите число: ");
    scanf("%d", &n);
    if (n > 0) {
        printf("Число положительное\n");
    }
}
```

```

        exit(1);
    } else if (n < 0) {
        printf("Число отрицательное\n");
        exit(2);
    } else {
        printf("Число равно нулю\n");
        exit(0);
    }
}

```



```

GNU nano 8.5
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    printf("Введите число: ");
    scanf("%d", &n);
    if (n > 0) {
        printf("Число положительное\n");
        exit(1);
    } else if (n < 0) {
        printf("Число отрицательное\n");
        exit(2);
    } else {
        printf("Число равно нулю\n");
        exit(0);
    }
}

```

Рисунок 2.5: Код checknum.c

2. Скомпилируем программу (рис. 2.6):

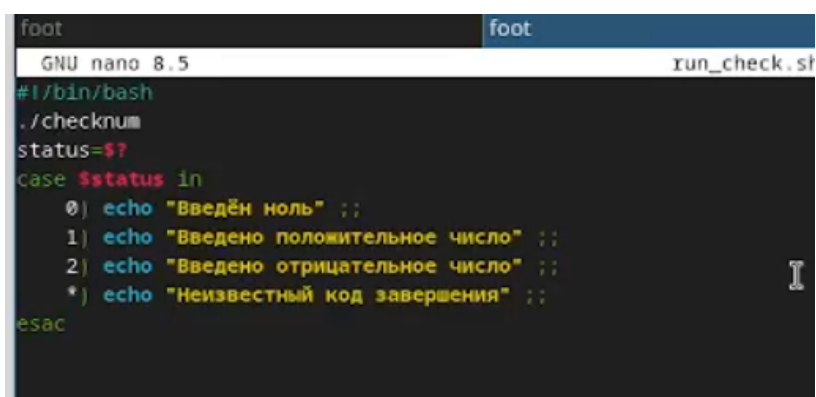
```
gcc checknum.c -o checknum
```

```
asivanova1@fedora:~/scripts$ nano checknum.c
asivanova1@fedora:~/scripts$ gcc checknum.c -o checknum
```

Рисунок 2.6: Компиляция checknum.c

3. Создадим скрипт run_check.sh для анализа кода возврата (рис. 2.7):

```
#!/bin/bash
./checknum
status=$?
case $status in
    0) echo "Введён ноль" ;;
    1) echo "Введено положительное число" ;;
    2) echo "Введено отрицательное число" ;;
    *) echo "Неизвестный код завершения" ;;
esac
```



```
GNU nano 8.5 run_check.sh
#!/bin/bash
./checknum
status=$?
case $status in
    0) echo "Введён ноль" ;;
    1) echo "Введено положительное число" ;;
    2) echo "Введено отрицательное число" ;;
    *) echo "Неизвестный код завершения" ;;
esac
```

Рисунок 2.7: Листинг run_check.sh

4. Сделаем файл исполняемым и протестируем на числах 8, -10, 0 (рис. 2.8 – 2.10):

```
./run_check.sh    # ВВОД 8
./run_check.sh    # ВВОД -10
./run_check.sh    # ВВОД 0
```

```
asivanova1@fedora:~/scripts$ chmod +x run_check.sh
asivanova1@fedora:~/scripts$ ./run_check
bash: ./run_check: Нет такого файла или каталога
asivanova1@fedora:~/scripts$ ./run_check.sh
Введите число: 8
Число положительное
Введено положительное число
```

```
asivanova1@fedora:~/scripts$ ./run_check.sh
Введите число: -10
Число отрицательное
Введено отрицательное число
```

```
asivanova1@fedora:~/scripts$ ./run_check.sh
Введите число: 0
Число равно нулю
Введён ноль
```

2.3 Задание 3. Создание и удаление нумерованных файлов

Разработаем скрипт `file_manager.sh`, который создаёт или удаляет файлы `1.tmp`, `2.tmp`, ..., `N.tmp` в зависимости от наличия ключа `-d`.

1. Листинг `file_manager.sh` (рис. 2.11):

```
#!/bin/bash

if [ $# -lt 1 ]; then
    echo "Использование: $0 [-d] N"
    echo "  без -d: создаёт файлы 1.tmp .. N.tmp"
    echo "  с -d: удаляет файлы 1.tmp .. N.tmp"
    exit 1
fi
```

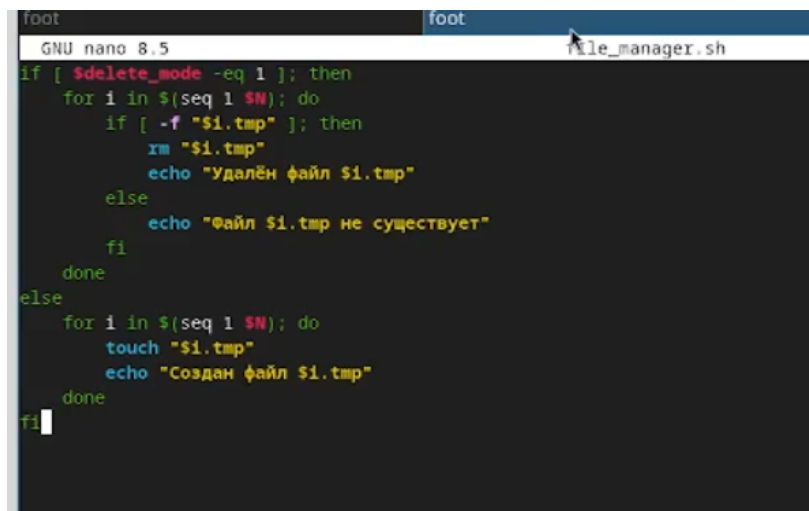
```

if [ "$1" = "-d" ]; then
    delete_mode=1
    N=$2
else
    delete_mode=0
    N=$1
fi

if ! [[ "$N" =~ ^[0-9]+$ ]]; then
    echo "Ошибка: N должно быть числом"
    exit 1
fi

if [ $delete_mode -eq 1 ]; then
    for i in $(seq 1 $N); do
        if [ -f "$i.tmp" ]; then
            rm "$i.tmp"
            echo "Удалён файл $i.tmp"
        else
            echo "Файл $i.tmp не существует"
        fi
    done
else
    for i in $(seq 1 $N); do
        touch "$i.tmp"
        echo "Создан файл $i.tmp"
    done
fi

```



```

foot
GNU nano 8.5 file_manager.sh
if [ $delete_mode -eq 1 ]; then
  for i in $(seq 1 $N); do
    if [ -f "$i.tmp" ]; then
      rm "$i.tmp"
      echo "Удалён файл $i.tmp"
    else
      echo "Файл $i.tmp не существует"
    fi
  done
else
  for i in $(seq 1 $N); do
    touch "$i.tmp"
    echo "Создан файл $i.tmp"
  done
fi

```

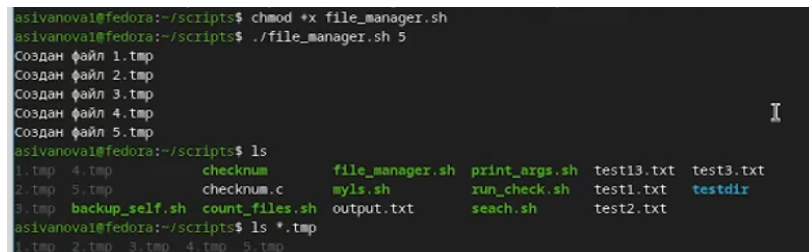
Рисунок 2.8: Листинг file_manager.sh

2. Сделаем файл исполняемым и создадим 5 файлов (рис. 2.12):

```

chmod +x file_manager.sh
./file_manager.sh 5
ls *.tmp

```



```

asivanovai@fedora:~/scripts$ chmod +x file_manager.sh
asivanovai@fedora:~/scripts$ ./file_manager.sh 5
Создан файл 1.tmp
Создан файл 2.tmp
Создан файл 3.tmp
Создан файл 4.tmp
Создан файл 5.tmp
asivanovai@fedora:~/scripts$ ls
1.tmp  4.tmp  checknum  file_manager.sh  print_args.sh  test13.txt  test3.txt
2.tmp  5.tmp  checknum.c  myls.sh          run_check.sh   test1.txt   testdir
3.tmp  backup_self.sh  count_files.sh  output.txt       seach.sh       test2.txt
asivanovai@fedora:~/scripts$ ls *.tmp
1.tmp  2.tmp  3.tmp  4.tmp  5.tmp

```

Рисунок 2.9: Создание файлов

3. Удалим созданные файлы (рис. 2.13):

```

./file_manager.sh -d 5
ls *.tmp

```

```

asivanova1@fedora:~/scripts$ ./file_manager.sh -d 5
Удалён файл 1.tmp
Удалён файл 2.tmp
Удалён файл 3.tmp
Удалён файл 4.tmp
Удалён файл 5.tmp
asivanova1@fedora:~/scripts$ ls *.tmp
ls: невозможно получить доступ к '*.tmp': Нет такого файла или каталога
asivanova1@fedora:~/scripts$

```

Рисунок 2.10: Удаление файлов

2.4 Задание 4. Архивация свежих файлов

Разработаем скрипт `backup_recent.sh`, который архивирует только файлы, изменённые за последние 7 дней в указанной директории.

1. Листинг `backup_recent.sh` (рис. 2.14):

```

#!/bin/bash

if [ $# -ne 1 ]; then
    echo "Использование: $0 <директория>"
    exit 1
fi

DIR="$1"

if [ ! -d "$DIR" ]; then
    echo "Ошибка: директория $DIR не существует"
    exit 1
fi

BACKUP_NAME="backup_$(date +%Y%m%d_%H%M%S).tar.gz"

```

```
find "$DIR" -type f -mtime -7 -print | tar -czf "$BACKUP_NAME" -T -  
  
echo "Архив создан: $BACKUP_NAME"
```

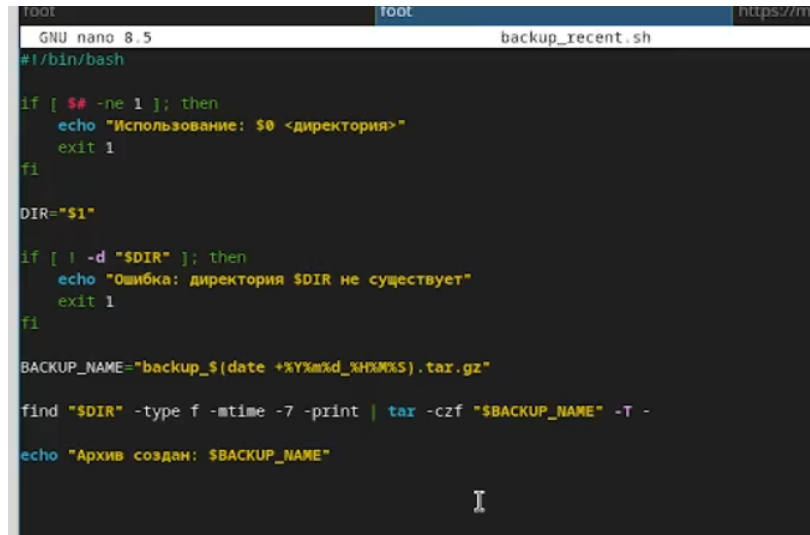


Рисунок 2.11: Листинг backup_recent.sh

2. Сделаем файл исполняемым и подготовим тестовую директорию с файлами разной давности (рис. 2.15):

```
mkdir testdir  
touch testdir/oldfile -d "10 days ago"  
touch testdir/newfile1  
echo "test" > testdir/newfile2
```

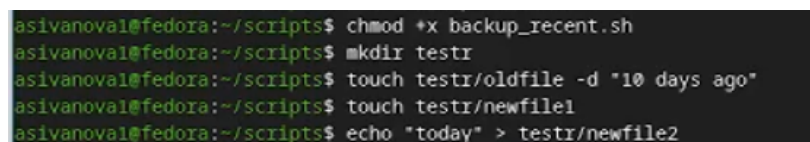
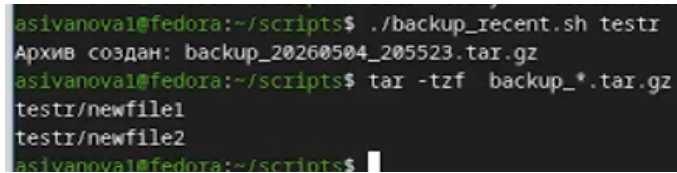


Рисунок 2.12: Подготовка testdir

3. Запустим скрипт и проверим содержимое архива (рис. 2.16):


```
chmod +x backup_recent.sh
./backup_recent.sh testdir
tar -tzf backup_*.tar.gz
```

A terminal window with a dark background and light green text. The prompt is 'asivanova1@fedora:~/scripts\$'. The first command is './backup_recent.sh testr', followed by the output 'Архив создан: backup_20260504_205523.tar.gz'. The second command is 'tar -tzf backup_*.tar.gz', followed by the output 'testr/newfile1' and 'testr/newfile2'. The prompt is partially visible at the bottom.

```
asivanova1@fedora:~/scripts$ ./backup_recent.sh testr
Архив создан: backup_20260504_205523.tar.gz
asivanova1@fedora:~/scripts$ tar -tzf backup_*.tar.gz
testr/newfile1
testr/newfile2
asivanova1@fedora:~/scripts$
```

Рисунок 2.13: Архивация и проверка

В архиве присутствуют только testdir/newfile1 и testdir/newfile2.

3 Контрольные вопросы

1. **Каково предназначение команды `getopts`?**

`getopts` используется для разбора опций командной строки в скриптах `bash`. Позволяет обрабатывать ключи с аргументами и без них, упрощая анализ параметров.

2. **Какое отношение метасимволы имеют к генерации имён файлов?**

Метасимволы (`*`, `?`, `[]`) служат для подстановки имён файлов (globbing) – замены шаблона на список существующих файлов.

3. **Какие операторы управления действиями вы знаете?**

`if`, `then`, `else`, `elif`, `fi`, `case`, `esac`, `for`, `while`, `until`, `break`, `continue`, `exit`.

4. **Какие операторы используются для прерывания цикла?**

`break` – немедленный выход из цикла, `continue` – переход к следующей итерации.

5. **Для чего нужны команды `false` и `true`?**

`true` всегда возвращает код 0 (истина), `false` – ненулевой код (ложь). Используются для организации бесконечных циклов или для задания условий.

6. **Что означает строка `if test -f man\${s}/\\\$i.\\\$s` в командном файле?**

Проверяет существование файла, имя которого формируется из пере-

менных `$s` и `$i`. Обратные слешы экранируют метасимволы, чтобы они воспринимались буквально.

7. Объясните различие между конструкциями `while` и `until`.

`while` выполняет тело цикла, пока условие истинно (код возврата 0). `until` – пока условие ложно (код возврата не 0). То есть `until` является антиподом `while`.

4 Вывод

Мы изучили основы программирования в оболочке ОС UNIX, научились писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.